

Master Degree in Statistics for Data Science
2022-2023

Master Thesis

“Machine learning-driven personalized marketing for Mercedes-Benz Company”

Pilar Pérez Piedra

Patricia Callejo Pinaro
Andrea Isabel Estaire Muñoz
Madrid, 09/2023

AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

Machine learning algorithms have attained a crucial position within leading companies, either to optimize internal processes or to design marketing campaigns. In the search of enhancing a specific marketing use case for the Mercedes-Benz company, an exhaustive exploration of the current solution, a classification LightGBM model, has been conducted. Once the required needs, and those that may arise in the future, have been identified, another machine learning method called Recommender System has been formulated and implemented. This approach effectively combines vehicle information and user preferences using the abilities of Deep Neural Networks. Comparing both solutions, the current one emerged as the easiest option to compute predictions in terms of coding, whereas the Recommender System has the potential to take the predicting process a stage further.

Keywords: Machine learning; Gradient boosting; LightGBM; Recommender System; Hybrid filtering; Neural Network.



Contents

1. Introduction	1
1.1. Context and motivation	1
1.2. Objectives.	3
2. Methodology and Tools	4
3. Data Exploration	7
3.1. Data description	7
3.2. Exploratory data analysis and data quality checks	9
3.3. Preprocessing steps and Feature Store	11
4. Review of Existing Approaches	13
4.1. Overview of the current state-of-the-art models	13
4.1.1 Integration of data and model input	15
4.1.2 LightGBM Regressor: Predicting the Next Purchase Date	16
4.1.3 LightGBM Classifier: Predicting the Next Best Vehicle	18
4.2. Performance metrics and evaluation criteria	19
4.3. Strengths and limitations of the model	21
5. Designing a Novel Approach: Recommender System	22
5.1. Introduction to Recommender Systems	22
5.2. Model description	24
5.3. Model architecture	25
5.3.1 Integration of data and model input	27
5.3.2 Hyper-parameters	29

5.4. Negative sampling approach	30
5.5. Workflow Implementation.	31
6. Evaluation and results of the novel approach	36
6.1. Performance metrics and evaluation criteria	36
6.2. Comparison with the existing model	37
6.3. Strengths and limitations of the model	39
7. Conclusions	40
7.1. Future lines of work	41
Bibliography	42

1.1 Context and motivation

Data science is the action of deriving valuable information from data, which can be carried out through several approaches. One of the most widely used today is known as Machine Learning, which combines computer science, maths and statistics fields. This approach consists of a set of algorithms that give computers the ability to learn from data on their own, without human intervention. Some examples include recommendation systems, regression, classification and time series, however, there is no universal winner that surpasses all others. The selection of the most suitable algorithm is completely linked to the problem itself.

In the business context, machine learning takes on extraordinary relevance. It empowered organizations to unlock information from large databases, enabling them to make informed and strategic decisions. This translates into significant advantages, such as optimizing internal processes, anticipating market trends and personalizing customer experiences. In summary, enhancing customer satisfaction while reducing costs and increasing benefits. A clear example of its applicability and potential can be found in the Mercedes-Benz company.

Mercedes Benz stands as a flagship entity in the automotive industry, noted for its constant pursuit of excellence and its position as a leader at the forefront of progress. In line with its commitment to innovation and performance, the German vehicle manufacturer closed the first half of 2023 with an attributable net profit of €7,508 million. This achievement reflects a significant increase of 13.8% compared to the same period in 2022 (Europa-Press, [2023](#)). These solid results are the product of a disciplined business

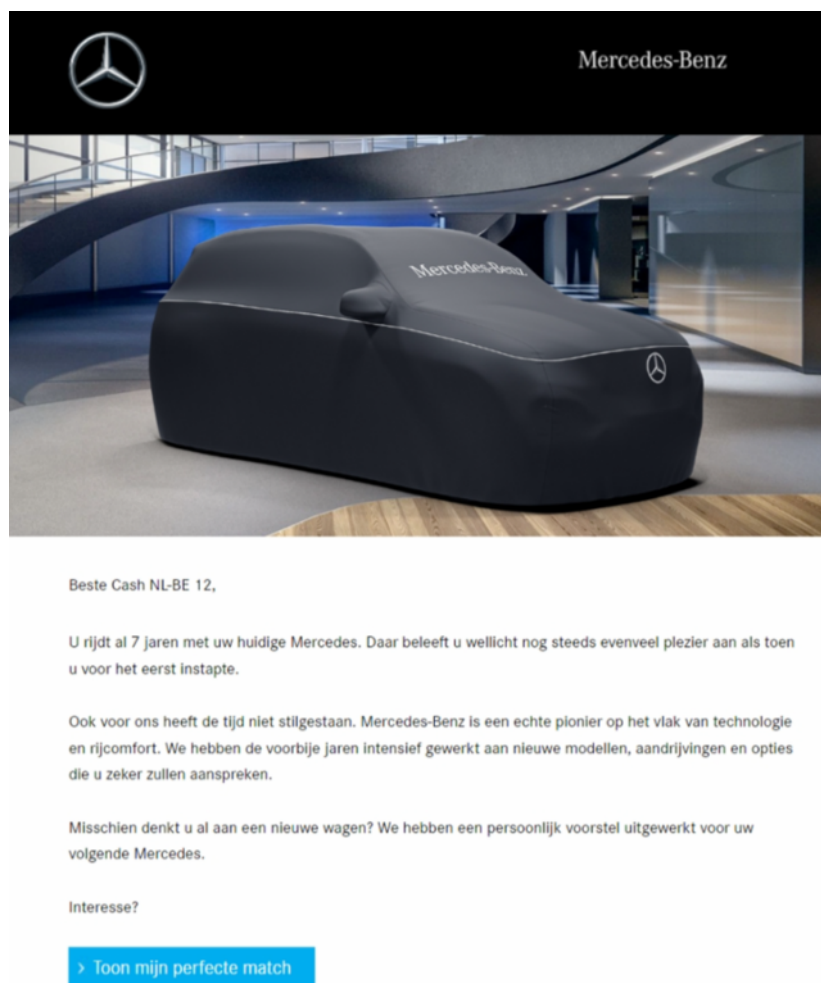
strategy that has been able to adapt with agility to a constantly changing environment.

A while ago, the company recognized the crucial role data science, in particular machine learning, should play in its strategy. This essential business resource had the power to increase efficiency and profits considerably as mentioned. An eloquent example of this strategy is found in the Mercedes-Benz Marketing and Sales team's use case known as NBx.

The NBx team focuses on expanding sales and increasing customer retention through email marketing campaigns. This approach is based on anticipating the likely date of the customer's next purchase and suggesting their ideal vehicle. Thus, an email will be sent, as illustrated in Figure 1.1, when the customer's next purchase date is close. In this email, an image of a car covered by a tarpaulin will be presented which, when clicked, will reveal the vehicle predicted in the use case. This strategy seeks to bring vehicles even closer to customers, increasing customer satisfaction with the brand and driving sales.

Figure 1.1

Mercedes-Benz german email.



1.2 Objectives

This Master's thesis addresses two primary objectives within the context of the NBx Marketing and Sales team at Mercedes-Benz.

Firstly, it seeks to present and explore the tools that are currently being employed by the NBx team. These tools are used for the purpose of generating predictions regarding the next most suitable vehicle for each customer. Through a detailed analysis, the methodologies and resources that drive these projections will be investigated, providing a complete understanding of their operation and application in strategic decision-making.

Secondly, the design and implementation of a novel machine learning approach. This tool, known as a deep learning-based Recommender System, has the essential purpose of improving the results obtained so far and taking the recommendations a step further. In particular, generating accurate and personalized recommendations not only concerning the vehicle's class and body type, but also extends to the Baumuster 6 vehicle configuration, encompassing class, body type, engine, fuel, and drive preferences for Mercedes-Benz customers.

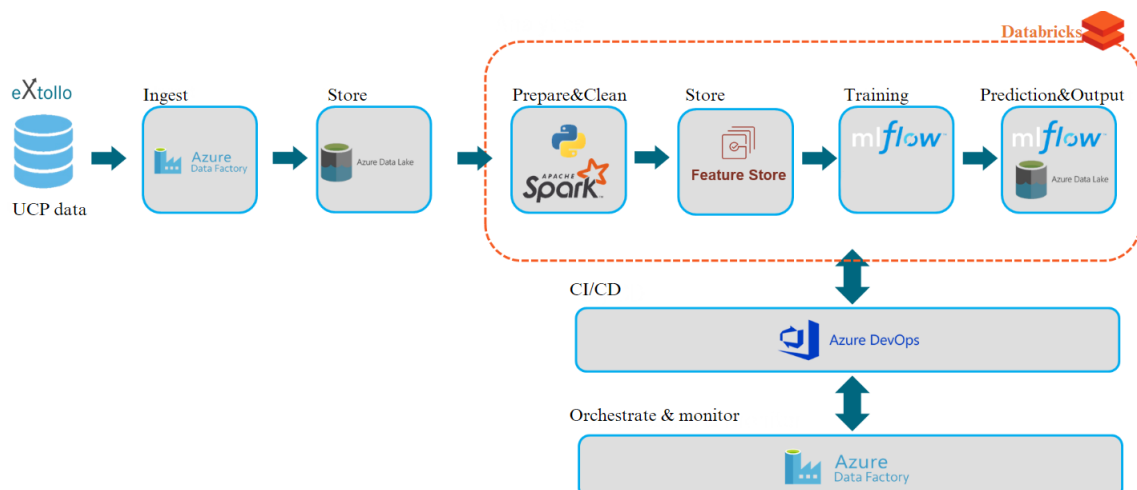
Methodology and Tools

Cloud computing has become increasingly popular in recent years as a versatile solution for a wide range of applications, from managing and processing large data sets to developing and deploying sophisticated machine learning models. One of the most widely used approaches due to its extensive range of services and scalability is **Microsoft Azure**, which provides a comprehensive suite of cloud-based services and tools to support data analytics, artificial intelligence, and other advanced computing tasks.

Mercedes-Benz leverages Microsoft Azure services through an internal platform called **eXtollo**. This platform stands as a strategic bridge that connects employees to the powerful set of tools available in Microsoft Azure but with an exclusively private approach. This methodology ensures the confidentiality and security of the data and operations.

Figure 2.1

Diagram of workflow tools.

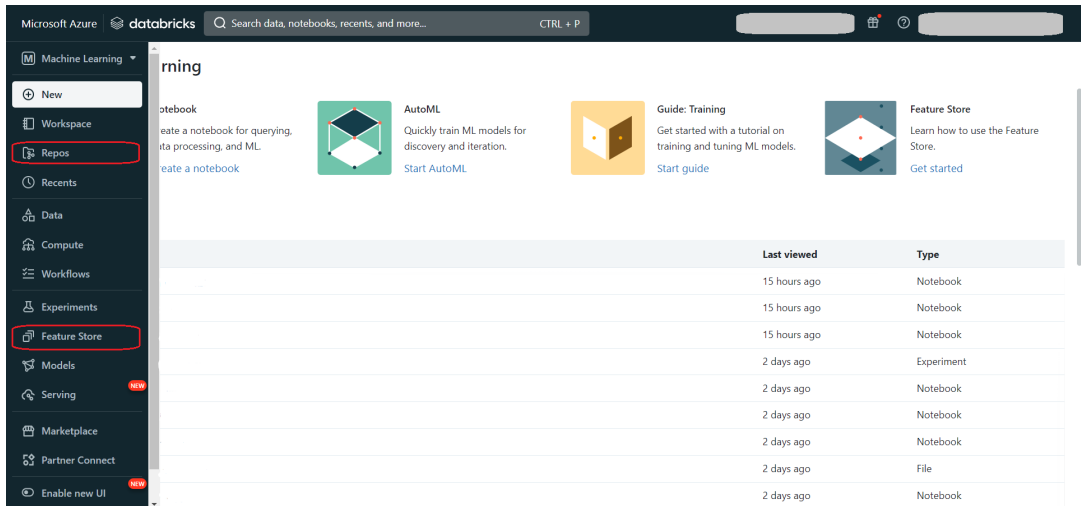


The comprehensive set of services that are going to be displayed in this section is illustrated in Figure 2.1. Thus, an overview of how the tools are integrated into the workflow to generate predictions for European customers is provided before going deeper into details.

The first step to address the task of enhancing the customer’s experience through car recommendations involves acquiring access to a reliable database. Mercedes Benz gathers information from its workshops, apps (Mercedes me) and dealerships to create a Unified Customer Profile(UCP) which is stored in eXtollo. The ingestion of this data to the Azure workspace relies on **Azure Data Factory**, a cloud-based data-integration service with pipelines that automate workflows, reducing the need for manual intervention and speeding up time. This tool stores data in the Azure environment **Azure Data Lake storage**, a cloud-based solution for file-based storage.

Once data has been successfully ingested and stored, the analytics section begins leveraging the capabilities of **Azure Databricks**, a cloud-based distributed platform for data processing and modeling built on Apache Spark. It provides a web-based user interface, an easy-to-use environment, that enables users to spend more time working effectively with data, and less time focused on managing clusters and infrastructure. This portal is integrated with the previous Azure services such that it allows working with the data stored in Azure Data Lake, the management of repositories and the creation of workspace resources (such as Spark clusters).

Figure 2.2
Azure Databricks Portal Interface.



The analytics process is implemented by Apache Spark on **Python notebooks**, which are stored in the Repos section of the Databricks Portal (see Figure 2.2). There, data undergoes an initial analysis, specifically targeting key aspects such as the variables distribution and the identification of missing values. Afterward, data quality checks are performed to ensure its cleanliness and readiness for modeling. Additionally, several data transformations take place, involving tasks like feature engineering, where new features

are derived from the existing ones to enhance the predictive power of the models. Finally, the resulting data is stored in **Feature Store**, a centralized repository that enables data scientists to find and share features. Figure 2.2 visually represents the accessibility of Feature Store within the Databricks platform.

Subsequently, data is split into train and test sets, allowing the development and evaluation of models. The models' design starts with essential tasks such as feature selection or parameter tuning, aiming to optimize the models' performance. To keep track of the entire machine learning lifecycle, an open-source platform called **MLflow** is employed within the Databricks framework. MLflow enables the storage of model versions along with their respective parameters and metrics, which are named experiments. This integration facilitates efficient management of the end-to-end machine learning process.

In the end, the model undergoes testing using the designated test set and the resulting metrics are registered together with the associated model in MLflow, using a model identification. Upon successful validation, the model is loaded from MLflow thanks to the identification number in order to generate predictions for customers. The resulting predictions are gathered into a prediction file, which is then stored in Azure Data Lake to send the marketing campaign.

Additionally, it is important to remark NBx is a team with more than 10 members developing together. Hence, to carry out an effective team collaboration and streamline the development and deployment processes, the Azure service called **Azure DevOps** plays a crucial role. This platform effectively manages the code repositories where those processes take place, tracking code changes. Indeed, those repos are linked to several pipelines created in Azure Data Factory in order to automate as much as possible the predicting process.

3.1 Data description

Mercedes-Benz European customers are the target of the current campaign. As a consequence, a data source that gathers several types of information such as customers' historical purchases, and personal and vehicle information is required. The selected one is UCP (Unified Customer Profile), a data source that unifies the information from all the European markets and which is accessible through the internal platform eXtollo.

The UCP database contains data from Mercedes-Benz workshops, dealers and the Mercedes-Benz app called 'Mercedes-me'. This data is structured in 10 tables, which are briefly described in Table 3.1. Out of these, 2 tables are organized at a customer level, meaning that there is an unique row per client, 2 at a vehicle level, with a unique row per vehicle sold, 2 at a customer-vehicle level, indicating that there is a row for each vehicle used by a customer. and the last 4 tables are organized in a way that includes multiple rows for the same customer and vehicle, capturing instances such as each time a client goes to the workshop with a particular vehicle, known as event-based level.

The data is updated monthly because legal requirements must be fulfilled, including the deletion of information for customers who revoke their consent to data usage. Therefore, it is important to remark on the version of data used for the campaign, June 2023. In this version, there was information for nine countries; the Netherlands, Czech Republic, Portugal, Germany, Italy, Sweden, Switzerland, Belgium and Luxembourg. However, the focus of this paper will be the Italian campaign, despite the analysis and the model comparison have been also done for the remaining ones.

Table 3.1*Tables description.*

Table name	Brief description	Row level
<i>private_customer</i>	Personal information such as the customer identification key (<i>ciam_id</i>), the birth date or the email address.	customer
<i>private_customer_derived_attributes</i>	Information to identify which customer have a finance/leasing contract or whether he/she has ever had one.	customer
<i>vehicle</i>	Vehicle specifications such as the vehicle identification keys (<i>fin</i> and <i>VIN</i>), vehicle class and bodytype, fuel type or engine code.	vehicle
<i>vehicle_order</i>	Information related with the first offer of the vehicle such as the order date, the offer price or the delivery date.	vehicle
<i>customer_vehicle</i>	Information relating customers with their purchases such as the vehicle and customer identification keys, the valid from/to (referring to which day the customer activates/deactivates the car), or the role.	customer - vehicle
<i>vehicle_finance</i>	Finance information such as the contract number, the financial product or the start/end date.	customer - vehicle
<i>digital_product_orders</i>	Digital extras information such as the product variant name (for instance, navigation services, smartphone integration or remote parking assist), the price of those extras or the expiry date.	event-based
<i>vehicle_damage</i>	Damage information gathered from Mercedes-Benz workshops such as the damage description (for instance, transmission oil or battery), the cost of the damage and whether it is covered by the warranty.	event-based
<i>agg_customer_vehicle_mileage</i>	Travelled distance information per year, including the year of measure, the total distance travelled in that year or the minimum and maximum mileage measured.	event-based
<i>vehicle_diagnosis</i>	Mileage records, including the date of the measurement, the mileage recorded and the unit of the measurement .	event-based

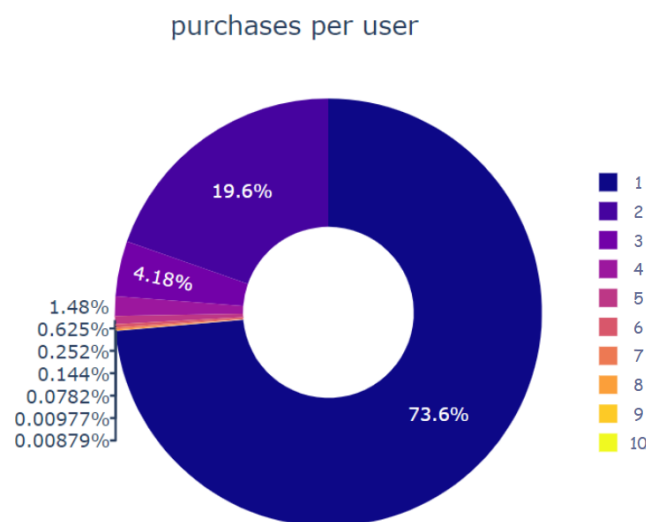
3.2 Exploratory data analysis and data quality checks

A comprehensive exploration of the columns and features of the tables has been undertaken, ensuring an exhaustive understanding of the data. However, to maintain conciseness and clarity, only key observations and specific features' values that are set to play a crucial role in shaping the models are going to be mentioned. Indeed, the illustrative examples will be drawn from the Italian (IT) market despite the fact that tables gather information from nine countries.

In the first place, historical purchase data points out that the initial recorded date traces back to seven years ago, specifically on '2016-10-17', when the Mercedes-me app was introduced. As a result, the number of purchases per customer registered within this time frame remains relatively low, i.e. the sequence of purchases has mostly one or two Mercedes-Benz purchases (see Figure 3.1).

Figure 3.1

Purchases per Italian user.



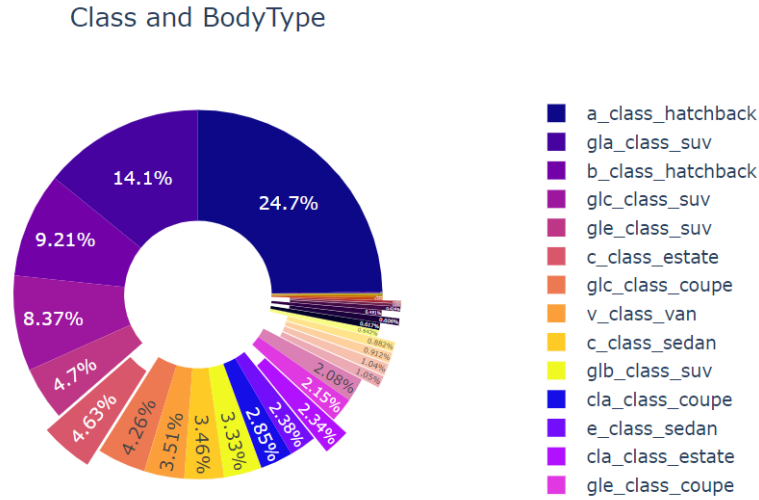
The fact that most customers have only registered one purchase might be a limiting factor for predicting the next best car. This implies that each customer is observed to be interested in only one or two specific car models. Indeed, a customer may have made a purchase in the past, but there is no history to determine whether he is satisfied with his choice or whether he subsequently chose to purchase a different vehicle. In the absence of a history to support this information, uncertainty is introduced that could affect the accuracy of the predicting model.

Furthermore, the aim of the current solution is to perform a classification problem, therefore the distribution of the target variable, the vehicles to be recommended (*class_bodytype*), should be analyzed and it is provided in Figure 3.2. The categories are not balanced, as 13 categories represent less than 2% of the data. This issue may have implications for the accuracy of the results and model-based decision making. It is therefore important to

consider class balancing strategies to improve the robustness and reliability of the model.

Figure 3.2

Class and Body type for Italian market. The legend includes only the top 14 most frequent vehicles out of the 38 combinations of class and body type.



In the second place, data quality issues were discovered during the exploration and the ones that required particular attention will be mentioned below.

On the one hand, inconsistencies in data standardization have been detected such as variations in the city (e.g. Roma, ROMA and roma), the street (e.g. ‘Via Paradiso’ and ‘paradiso’) and the name (e.g. Michele and michele). Besides, invalid values were detected for the email column of the Italian customers such as nomail@nomail.com, as a consequence, the predictions will be delivered to nowhere.

Moreover, unfeasible values for several features were detected. For example, negative prices were found in tables that include information of extra digital products, such as smartphone integration. These rows were compared to others that include similar digital products, concluding that the negative sign is a typographical mistake because the absolute values for the same digital product coincide.

On the other hand, several identification keys were detected for the same customer, i.e. there exist several *ciam_id* numbers for one client. The following table shows this issue as the *Ciam_id_count* per market is different from the *Cust_count*, the number of customers. The criteria for assuming that several rows correspond to the same customer were identical values for birth date, first name, last name, and city. Additionally, the *Cust_with_diff_ids* column represents the number of clients that have several identification keys, which may diverge from the difference between *Ciam_id_count* and *Cust_count* if more than two *ciam_id* numbers are associated to the same customer.

Table 3.2

Statistics for the data quality issue: Several identification keys for the same customer.

Market	Ciam_id_count	Cust_count	Cust_with_diff_ids	Cust_with_diff_ids_perc
IT	89797	80647	150	0.19%

Notwithstanding the relatively small percentage that this issue represents within the market campaign, the data source should fix it as soon as possible. The reason is that it leads to sending several predictions for the same customer. As a result, those customers will be disturbed more than once and inconsistencies in these predictions may arise.

3.3 Preprocessing steps and Feature Store

Once data has been thoroughly analyzed and the challenges clearly identified, a sequence of transformations is implemented in order to mitigate their impacts on the final model.

The current solution to address the campaign requires that those combinations of class and body type that have a limited presence in the data are substituted. This is done by incorporating more representative alternatives using a *class_bodytype* dictionary, provided by the company, that relates similar vehicles. For instance, in the Italian context, the *ege_class_sedan* has less than 100 acquisitions therefore it will be replaced by *e_class_sedan*.

Significant discrepancies in values that convey the same meaning tried to be standardized. As an example, the change to lowercase was applied to the values of the above-mentioned columns in order to solve issues related to case differences. Those variations with low representation, such as ‘Via Paradiso’ and ‘paradiso’, were not handled, as dealing with all possible situations in a real database is practically unfeasible and time-consuming. In addition, negative price values were replaced by their absolute value equivalents, as they were found to be consistent for the same type of product.

Duplicated records were removed in order to have a single entry per customer and vehicle in the purchase history data. Nevertheless, data still have customers with different identification keys due to addressing this situation requires, from a business perspective, the formulation of a uniform criterion for its identification. Given its small percentage, its resolution will be postponed until the database handles it, which, in turn, could introduce some uncertainty into the model.

Feature engineering tasks are also performed to create new columns that could provide valuable information for the predicting campaign. For instance, there is a column called *vehicle_type* that contains multiple vehicle specifications together. In this way, the information would be considered as a set of elements and the model would lose the opportunity to analyze the combinations individually. For this reason, the column was divided into additional columns, as illustrated in Table 3.3.

Table 3.3

Data example. The column vehicle_type is provided by UCP and the remaining ones are the result of the feature engineering process.

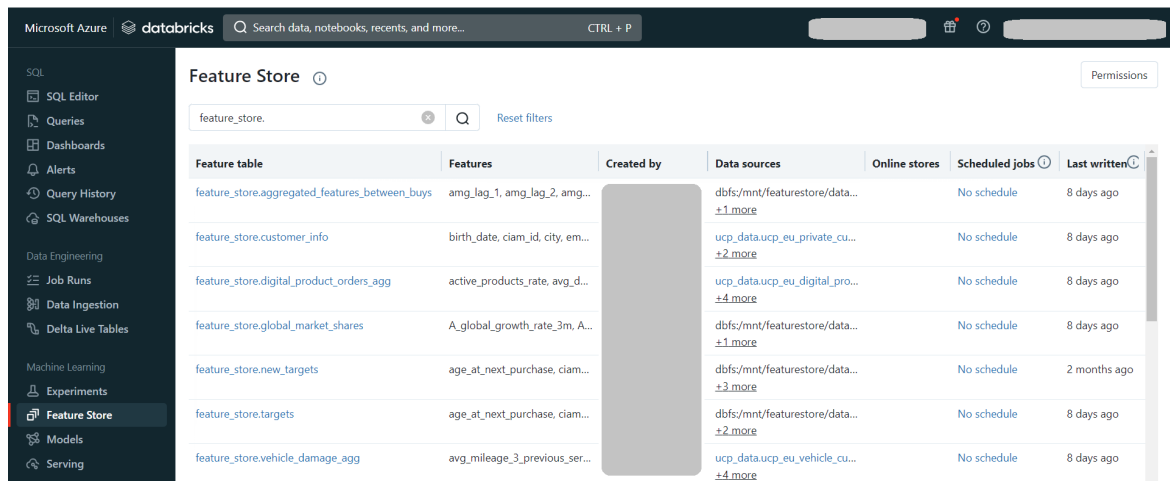
vehicle_type	class	engine_code	fuel_type	drivetrain
A 250 4MATIC	A	250	null	4MATIC
E 220 d 4MATIC	E	220	d	4MATIC
C 200	C	200	null	null

Another example is the creation of columns that aggregate clients' transactional details such as the pre-purchase count of previously acquired vehicles per customer, the pre-purchase count of vehicles sold, or the age at which the acquisitions were made. Achieving this task was greatly facilitated by leveraging the PySpark functionality of window functions.

Up to this point, several transformations required for the construction of a reliable predictive model have been applied, therefore, in order to preserve the 'clean' data, a Databricks component called Feature Store comes into play. This tool can be conceived as a centralized repository designed for the efficient storage and management of the features used in the development of machine learning models. It is also extremely useful to keep track of the table history creation and to schedule automated jobs. (Databricks, 2023)

Figure 3.3

Feature Store.



The screenshot shows the Databricks Feature Store interface. The left sidebar contains navigation options: SQL, SQL Editor, Queries, Dashboards, Alerts, Query History, SQL Warehouses, Data Engineering, Job Runs, Data Ingestion, Delta Live Tables, Machine Learning, Experiments, Feature Store (selected), Models, and Serving. The main panel displays a table of feature tables with columns: Feature table, Features, Created by, Data sources, Online stores, Scheduled jobs, and Last written. The table lists several feature tables, including 'feature_store.aggregated_features_between_buys', 'feature_store.customer_info', 'feature_store.digital_product_orders_agg', 'feature_store.global_market_shares', 'feature_store.new_targets', 'feature_store.targets', and 'feature_store.vehicle_damage_agg'. Each row shows the features, data sources, and the last written time.

Feature table	Features	Created by	Data sources	Online stores	Scheduled jobs	Last written
feature_store.aggregated_features_between_buys	amg_lag_1, amg_lag_2, amg...		dbfs/mnt/featurestore/data... +1 more		No schedule	8 days ago
feature_store.customer_info	birth_date, ciem_id, city, em...		ucp_data.ucp_eu_private_cu... +2 more		No schedule	8 days ago
feature_store.digital_product_orders_agg	active_products_rate, avg_d...		ucp_data.ucp_eu_digital_pro... +4 more		No schedule	8 days ago
feature_store.global_market_shares	A_global_growth_rate_3m, A...		dbfs/mnt/featurestore/data... +1 more		No schedule	8 days ago
feature_store.new_targets	age_at_next_purchase, ciem...		dbfs/mnt/featurestore/data... +3 more		No schedule	2 months ago
feature_store.targets	age_at_next_purchase, ciem...		dbfs/mnt/featurestore/data... +2 more		No schedule	8 days ago
feature_store.vehicle_damage_agg	avg_mileage_3_previous_ser...		ucp_data.ucp_eu_vehicle_cu... +4 more		No schedule	8 days ago

Thus, 10 tables are created in Feature Store, one for each of the ucp tables previously described in the data description section. The corresponding ucp data is stored there once the required transformations have been applied. It is important to remark that additional tables are also created in Feature Store, for example, the 'Targets' table, which is used to make joins with the tables mentioned above. However, these tables are only necessary for the current model that is in production in the NBx team, therefore a detailed description of these tables will be provided in its corresponding section 4.1.1.

Review of Existing Approaches

The main goal of the NBx team is to increase customer satisfaction and boost vehicle sales by offering personalized recommendations. However, it is crucial to avoid annoying customers by constantly overloading them with suggestions. In this manner, the recommendations could potentially backfire, as they might lead to the withdrawal of user consent for data processing, or worse, brand disengagement. For this reason, the team has developed two complementary models to address this situation.

Firstly, a regressor model has been implemented to predict the next best purchase date for users. In this way, recommendations will only be sent to them when it is anticipated that they may be interested in purchasing a new vehicle.

Secondly, for those users whose next purchase date is within the next three months, a classification model has been created to predict what might be the next vehicle that will attract their interest.

In the following, the structure and results of both models will be presented, since in both cases LightGBM technology has been used for their development and similar procedures have been followed. Nevertheless, it is crucial to remark that the objective of this paper is just focused on the next best car model, trying to enhance its performance using a deep learning-based approach of a Recommender System.

4.1 Overview of the current state-of-the-art models

Up to this point, NBx campaigns are based on the application of predictive models using LightGBM, a gradient-boosting framework that uses tree-based learning algorithms.

Let's delve into these algorithms offering insights into their mechanisms and distinctive features.

The term "boosting" refers to a technique in machine learning that focuses on combining multiple weak models, known as weak learners, to generate a more robust and accurate model. Those "weak learners" are algorithms that perform slightly better than random guessing. The boosting process involves several steps. Initially, all observations in the training set are assigned uniform weights and a weak learner is trained. Then, the weights of the observations are adjusted, those observations that were incorrectly classified by previous learners receive a higher weight, and the process is repeated several times. In this way, the learning process is focused on correcting previous errors. Finally, the output of the weak learners is combined and a final prediction is obtained,

$$\hat{F}(x) = \sum_{j=1}^N \lambda \hat{F}_j(x),$$

where λ is the shrinkage parameter, N the number of weak learners and $\hat{F}_j$ is the prediction function of the weak learner j .

Gradient Boosting is a modified version of the boosting technique that replaces the traditional approach of using the error from each weak learner with the utilization of gradient descent. This adjustment aims to minimize a pre-defined loss function by sequentially incorporating multiple weak learners. At each iteration k , residuals are computed, which are defined by the gradient of the loss function and represent the direction in which the model needs to improve, and the prediction function is updated as:

$$\hat{F}_{k+1}(x) = \hat{F}_k(x) - \alpha_k h_k(x),$$

where $\hat{F}$ is the prediction function, h is a weak learner computed from gradient residuals and α is the learning rate (Friedman et al., 2001).

However, conventional implementations of Gradient Boosting can become computationally expensive, as they must examine all instances and features to estimate information gains. This is where LightGBM comes into play. LightGBM is an implementation of Gradient Boosting employing two additional techniques called Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB). GOSS focuses on selecting critical instances for learning and discarding those that are not as informative. EFB bundles similar features to accelerate the search for better splits in trees. The detailed implementation steps of the algorithms are available for consultation in the source referenced as Ke et al., 2017.

These modifications make LightGBM especially effective on large datasets, offering the following advantages: faster training speeds, reduced memory usage, improved accuracy and support for parallel, distributed, and GPU-based learning techniques (Microsoft LightGBM Contributors, 2023).

4.1.1 Integration of data and model input

The working procedure to implement the predictive models starts with the preparation of the model input. On the one hand, additional summary tables are created in Feature Store, aggregating information from the previous ones, in order to enhance model performance. For instance, the table *Vehicle_market_shares* is built computing the proportion of vehicles with a specific class or body type sold in a given market at a specific period of time, month-year.

On the other hand, the model input requires a single set of data. Then, a table called ‘Targets’, which contains the necessary keys to aggregate the information coming from the other tables in Feature Store, is also generated. The keys are the columns: *ciam_id* (to relate the tables at customer level), *exec_date* (to join identical versions of the tables, such as the June 2023 version), *market* (to match according to market), *year_month* (to join identical time periods), *fin* (for matching by vehicle), as well as *valid_from* (to match according to the precise time the customer connect the vehicle). The event-based data, such as each time customers buy a digital product or go to the workshop, is quite new and it is still not fully integrated into the current solution, only those that occur at the same time of a purchase are considered, joining the information by the key *valid_from*.

Figure 4.1

Illustrative example of some tables from Feature Store with its corresponding keys. The model’s input data frame will be computed joining to the Targets the information of the remaining tables, using the key columns highlighted in bold.

Customer_info							
ciam_id	market	exec_date	salutation	birth_date	city	email	
00ab79e	IT	202306	MR	1976-05-19	roma	michael@gmail.com	
00df18b	IT	202306	MRS	1987-12-02	roma	valentino@web.com	
Vehicle_info							
fin	exec_date	class	bodytype	fuel_type	drivetrain	engine	amg
W1A45	202306	C	coupe	petrol	4x2	220	0
W1A29	202306	A	hatchback	petrol	4x2	180	0
W1A11	202306	C	estate	petrol	4x2	300	1
Vehicle_relationship							
ciam_id	exec_date	market	fin	valid_from	valid_to	role	age_at_purchase
00ab79e	202306	IT	W1A45	2016-12-05	2020-03-24	OWNER	40
00ab79e	202306	IT	W1A29	2020-03-24	2023-05-01	OWNER	44
00ab79e	202306	IT	W1A11	2023-05-01	null	USER	47
00df18b	202306	IT	W1A45	2020-04-20	null	OWNER	33
Vehicle_market_shares							
year_month	exec_date	market	coupe_market_share_y		sedan_market_share_y	estate_market_share_y	
2020-03	202306	IT	0.1224		0.2243	0.1669	
2023-05	202306	IT	0.1360		0.2268	0.1609	
2023-06	202306	IT	0.1343		0.1953	0.1601	
Targets							
ciam_id	exec_date	market	fin	valid_from	year_month	next_class_body	next_purchase_date
00ab79e	202306	IT	W1A45	2016-12-05	2020-03	A_class_hatchback	2020-03-24
00ab79e	202306	IT	W1A29	2020-03-24	2023-05	C_class_estate	2023-07-01
00ab79e	202306	IT	W1A11	2023-05-01	2023-06	null	null
00df18b	202306	IT	W1A45	2020-04-20	2023-06	null	null

Figure 4.1 illustrates a hypothetical example of two different customers. The table *Customer_info* gathers one row for each, including their personal information and the identification key *ciam_id*. The *Vehicle_info* contains specifications of three vehicles, however, the *Vehicle_relationship* table should be considered to identify which customer purchased which vehicle. Indeed, this table shows that the first customer bought a C-class coupe in 2016, an A-class hatchback in 2020 and currently he is driving a C-class estate. Once he had deactivated the C-class coupe in 2020, the second customer bought the vehicle, as the vehicle identification *fin* coincides, and he is still driving it.

The *Vehicle_market_shares* table contains information on how popular each body-type is per market at a specific period, month - year, to add information on the market preferences at the time the customer makes its following purchase. Finally, the *Targets* table includes all the keys highlighted in bold of the remaining tables plus the target columns for the predicting models, the next purchase date and the next class bodytype. Here, the *year_month* of each row is understood as the period of time at which the customer made the following purchase, and if no following purchase is found, then the current month.

As a consequence, the model input will be the resulting data frame of adding to the *Targets* table the information of the remaining tables, using the keys to perform the join. In this way, the model will try to discern patterns that relate the columns' information with the target one.

4.1.2 LightGBM Regressor: Predicting the Next Purchase Date

The objective is to predict when customers will make their next purchase, in order to be able to launch a timely marketing campaign, i.e. when they are interested in purchasing a new vehicle. To achieve this, the number of months between the date a customer purchased a vehicle (*valid_from*) and the date of their next purchase (*next_purchase_date*) is computed, called *time_until_next_purchase*. With this information, a regression model based on LightGBM is built, specifically, using the `LGBMRegressor` function from the `lightgbm` python module.

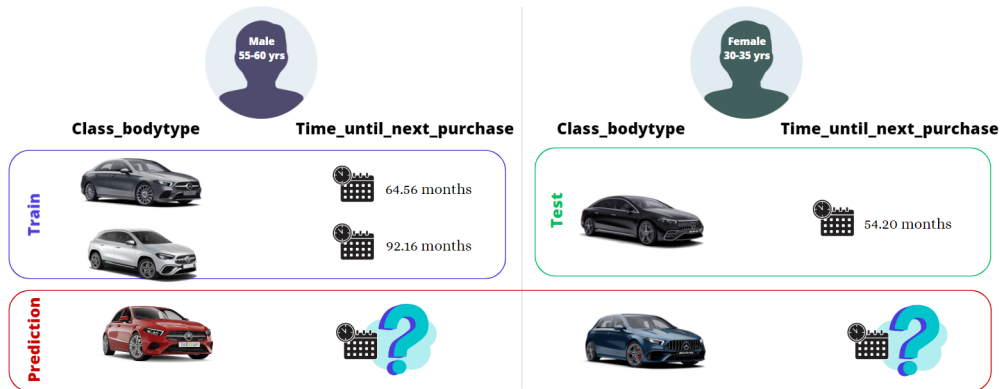
Nevertheless, the resulting data frame of the previous section should be firstly split into train, test and prediction sets. The last group is the easiest one to compute, all the rows with *time_until_next_purchase* empty don't work to train or test the algorithm because they don't have a target value to learn from or compare with. They are the rows that require a prediction, then they will make up the prediction set. The remaining rows will be split at a customer level, i.e. 20% of customer identifiers will be randomly selected to test the algorithm. As a result, all the rows with the selected ids will make up the test group, and the remaining ones the train set.

A hypothetical example to illustrate the situation is provided by the Figure 4.2. The left-hand side customer has 3 rows within the data frame, one for each *Class_bodytype* he bought. The last one requires a prediction because there is no further purchase, then this

row will be kept for model prediction. Moreover, the customer has not been selected for testing, therefore the remaining two rows will be used to train the algorithm. In contrast, the right-hand side customer has 2 rows within the data frame, and he has been selected for testing. As a consequence, the last row will be kept for model prediction and the previous one will be used to test the model.

Figure 4.2

Illustrative example of the splitting process into train, test and prediction sets.



To improve the performance of the algorithm, optimization techniques are implemented using the train set. Firstly, a feature selection is carried out through a feature selector. At this stage, columns with very low variability, i.e., those with 99% or more identical values, are excluded. In addition, highly correlated columns, i.e., those with a correlation greater than 99%, are discarded. Subsequently, a sequential feature selector is employed, using an LGBMRegressor as the estimator and considering the negative root mean squared error as the score. The SequentialFeatureSelector function, from the mlexend python library, was the chosen tool to perform the selection.

Secondly, hyperparameter tuning is also performed in order to enhance the results as much as possible. For this, the Hyperopt python library is used, defining the following search space:

```

1 search_space = {
2     'learning_rate':    hp.loguniform('learning_rate', -3, 0),
3     'max_depth':       scope.int(hp.quniform('max_depth', 4, 100, 1))
4     ,
5     'min_child_weight': hp.loguniform('min_child_weight', -1, 3),
6     'colsample_bytree': hp.uniform('colsample_bytree', 0.3, 1.0),
7     'subsample':       hp.uniform('subsample', 0.8, 1),
8     'num_leaves':      scope.int(hp.quniform('num_leaves', 30, 150,
9     1)),
10    'seed': 123
11 }

```

Once this process is completed, the LGBMRegressor model is trained with the selected features and the optimal hyperparameters. The resulting model is registered using

an identification number in MLflow. This is extremely useful because the model could later be loaded using the identification number and the selected features, and hyperparameters, could be checked through the Models section in the Databricks portal.

Afterward, the model is loaded to predict the test set. The resulting test predictions are then compared with the true values and several metrics are computed to provide an initial assessment of the model's expected performance. Once they are validated, the last step of the process is to generate predictions for the rows that *time_until_next_purchase* is empty, the prediction group. At this point, the Feature Store becomes relevant, since, by registering the features used in the model, the Feature Store can add to the Targets table only the necessary columns selected by the model. This process is known as feature store lookup. In this way, predictions can be generated easily and efficiently just by providing the model's identification number and the Targets table to a Feature Store function.

4.1.3 LightGBM Classifier: Predicting the Next Best Vehicle

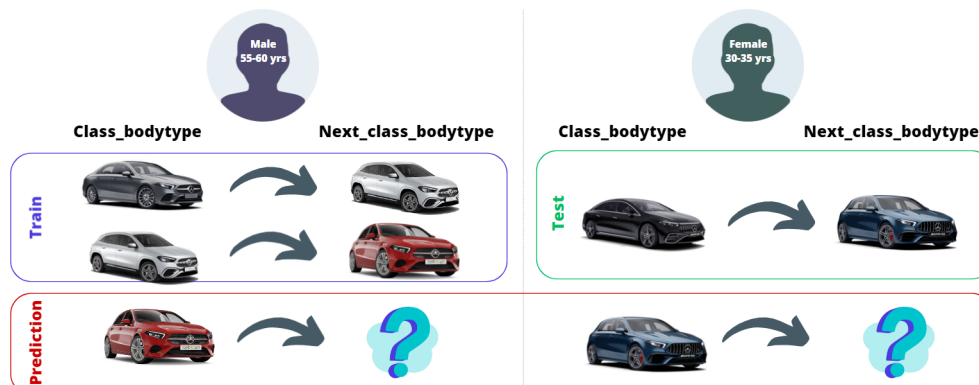
The focus in this case is on predicting the next most suitable car for those customers whose future purchase date is within the next 3 months. For this purpose, a classification model based on the LightGBM algorithm is created.

Firstly, due to the multiple combinations of class and body type, it is required to rename those less frequent combinations with other similar combinations more represented in the data, so that the model does not present a significant imbalance. Specifically, all combinations with less than 20 records are renamed. In addition, vehicle models that are not available in specific markets are excluded from the analysis.

Secondly, the data frame split should also take place. In this case, a similar procedure is followed, using the target column *next_class_bodytype*. On the one hand, those rows without the target column information are kept for prediction. On the other hand, 20% of customers are randomly selected such that their rows will make up the test set, and the remaining ones the train group. A hypothetical example could be seen in Figure 4.3.

Figure 4.3

Illustrative example of the splitting process into train, test and prediction sets.



Thirdly, an optimization structure similar to the one applied in the LGBMRegressor is also used. This time, the feature selection process is established based on the accuracy and the estimator used is the LGBMClassifier. Once the optimal features are identified, let's proceed with the hyperparameter tuning, using the following search space:

```

1  search_space = {
2      'n_estimators':      hp.choice('n_estimators', range(100, 400, 50))
3      ,
4      'learning_rate':     hp.loguniform('learning_rate', -3, 0),
5      'max_depth':         scope.int(hp.quniform('max_depth', 4, 100, 1))
6      ,
7      'min_child_weight':  hp.loguniform('min_child_weight', -1, 3),
8      'colsample_bytree':  hp.uniform('colsample_bytree', 0.3, 1.0),
9      'subsample':         hp.uniform('subsample', 0.7, 1),
10     'num_leaves':         scope.int(hp.quniform('num_leaves', 30, 150,
11     1)),
12     'seed': 123
13 }

```

The training and evaluation of the LightGBM Classifier model is carried out and saved in MLflow, together with the selected features and the optimal hyperparameters. As a consequence, the records with *next_class_bodytype* empty are finally predicted using the created model and the feature store lookup.

4.2 Performance metrics and evaluation criteria

To begin with, metrics related to the regression model are detailed, the one used to predict the number of months that will elapse until each customer's next purchase, starting from the activation date *valid_from*. Denoting n the number of test rows, y_i the number of months until next purchase of observation i and $\hat{y}_i$ the predicted number of months until next purchase for the i record,

- Mean Absolute Error:

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|.$$

- Root Mean Square Error:

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}.$$

- Explained variance score:

$$\text{Explained Variance Score} = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i - \overline{(y - \hat{y})})^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

- Coefficient of determination :

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}.$$

These measures assess both the overall predicting power of the model and its ability to capture variability in the data, providing a complete picture of its performance.

Table 4.1

Repurchase date model metrics for Italian data.

Training size	MAE	RMSE	Explained variance	r^2	Runtime
14104	13.52	16.8	0.294	0.294	6min

Overall, the regression model designed to predict the next purchase date presents a high absolute error. Table 4.1 particularly shows an absolute error in the Italian market of about 13 months, which implies a significant variability in the predictions. This translates into an average discrepancy of approximately one year in predicting the next purchase. However, it's worth noting that this variability is partly due to cases where some customers purchase vehicles at very short intervals due to the simultaneous ownership of multiple cars. Moreover, the *valid_from* column reflects the time of vehicle activation by the customer, meaning that the actual purchase could have occurred even a year earlier, but factors such as inventory availability may have delayed delivery. As a result, anticipating the forecast by up to 12 months is not necessarily considered unfavorable in this context.

In a similar vein, metrics related to the classification model that aims to predict the best class and body type of each customer's next purchase are presented. Since classes of the target variable are unbalanced, the evaluation of the accuracy of the first three recommendations is not enough, a metric that handles the unbalance is required. The selected one has been the F1 score weighted, which takes into account the proportion of each class in the data. This ensures that the assessment is more fair and that dominant classes are not favored.

$$F1_{weighted} = \frac{\sum_{i=1}^N w_i \cdot F1_i}{\sum_{i=1}^N w_i}, \quad F1_i = \frac{2PRE \cdot REC}{PRE + REC}, \quad PRE = \frac{TP}{TP + FP}, \quad REC = \frac{TP}{TP + FN},$$

where TP denotes the true positives, FP are the false positives, FN are the false negatives and w_i is the weight associated with the class i .

Table 4.2

Class and Bodytype model metrics for Italian data.

Training size	Acc top1	Acc top2	Acc top3	f1 weighed	Runtime
14104	0.261	0.393	0.489	0.247	4min

Table 4.2 shows that the classification model developed to forecast the most suitable car for Italian customers predicts correctly 26% of the initial recommendations. While this percentage may seem modest at first glance, it is relevant to keep in mind the considerable diversity of possible combinations. In this perspective, despite there is still room for improvement, achieving an accuracy level of 26% is positively valued from a business point of view. In addition, nowadays only the top 1 prediction is delivered, however, future campaigns will also include top 2 and 3 predictions, which achieved nearly 50% of correct recommendations.

4.3 Strengths and limitations of the model

The LightGBM stands out for several advantages that contribute to its popularity and success in a variety of applications. One of the highlights is its ability to achieve high-quality results with exceptional efficiency. Thanks to its tree-based structure and optimized learning algorithm, the LightGBM can handle large volumes of data efficiently. Through proper sampling and weighting techniques, the model can mitigate problems associated with underrepresented classes compared to other boosting algorithms, thus improving its ability to correctly identify examples of all classes.

However, LightGBM presents certain limitations that must be considered when implementing this model. One of the main concerns relates to its sensitivity to hyperparameters. The performance of the model and the quality of the predictions may depend significantly on how these parameters are set. Consequently, a careful and thorough hyperparameter tuning process is required to maximize the effectiveness of the model. In addition, as the number of combinations in the data increases, LightGBM may require significant computational resources and longer processing times. This can affect the scalability and speed of training on complex and large datasets.

Moreover, the train-test split technique used for the algorithm leads to only learning patterns from a subset of customers, the ones in the train set. As a result, the testing procedure could be understood as how well the model performs extrapolating patterns from the train customers to predict the next best car of customers in the test set. However, this approach does not provide an accurate assessment of the prediction quality because the prediction set contains both customers whose previous purchase history has been also learned by the model and customers about whom it has no prior data. Indeed, it is also relevant to mention that the model can not be trained or tested with customers who only bought one car because there is no target column for them, *next_class_bodytype*. Given that 73.6% of Italian customers have only 1 purchase, see Figure 3.1, the model can use the data from only 26.4% customers for training and evaluation.

Designing a Novel Approach: Recommender System

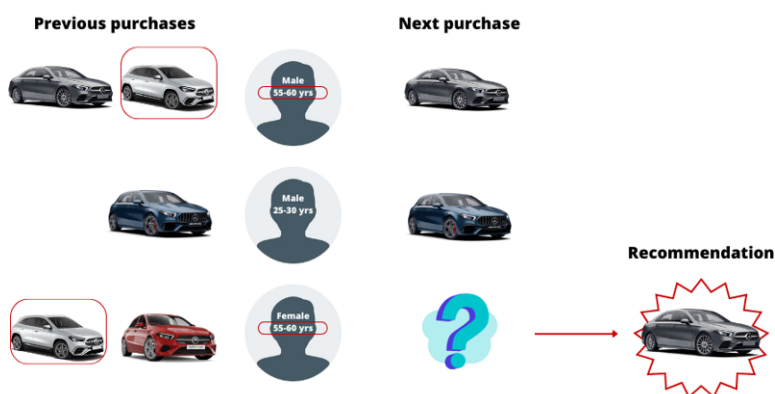
5.1 Introduction to Recommender Systems

Recommender Systems (RSs) are algorithms designed for the purpose of suggesting either an item or a service, using information relating to the user to whom the recommendation is intended to be made, to the object to be recommended, or even to both instances together.

In the first case, where RS uses the user data when processing information for recommendation, the tool is classified in the collaborative filtering category (see Figure 5.1). For example, by accessing user profiles in an online music shop, the Recommender System gains access to a wide range of information about the user, such as their age, geographic location, and the songs they have previously purchased. Using this information, the system has the ability to identify users who share similar characteristics with each other and, based on this, suggest songs purchased by other similar users.

Figure 5.1

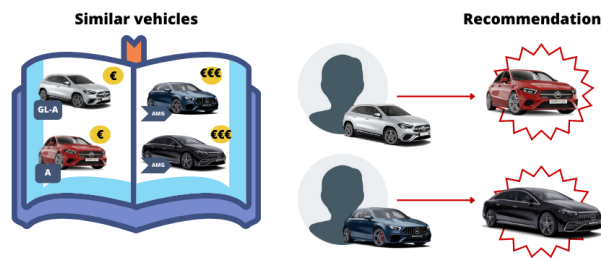
Collaborative filtering example.



In the second case, where RS lies on the item data to generate recommendations, the tool is classified in the content-based category (see Figure 5.2). For instance, consider an user searching for a new computer in an online shop. When the user browses a particular computer model (item) on the website, the Recommender System stores information about that model and proceeds to scan a database of other computers with similar characteristics, such as price, CPU speed and memory capacity. The result of this search and analysis process is translated into a series of recommendations addressed to the user, presenting him options that are similar to his initial interest (Portugal et al., 2018).

Figure 5.2

Content-based filtering example.



The third category is a combination of the previous two cases, an hybrid filtering approach, recommending items based on the user and the item data itself. Considering that data about users and items is accessible within the database described in section 3, opting for this choice is most suitable. The strength of hybrid filtering lies in its capacity to handle limited data, to introduce a variety of recommendations, to adjust to alterations, and to tackle the challenge posed by new users who lack substantial information. This makes it a flexible and effective strategy to enhance the user experience and to provide recommendations that are highly tailored to individual needs and preferences.

Despite the current appearance of RS as a novel tool, garnering attention from major companies such as Netflix, Google, and Amazon, its origin dates back to 1992, with the emergence of Tapestry. It was based on users tagging and classifying content, then preference profiles were created to recommend similar content to other users with similar interests.

The exploration of this approach as opposed to the LightGBM classification model arises from the distinctive characteristics of Recommender Systems. These systems are also notable for their ability to enable novelty detection, to recommend a wide variety of personalised choices and to address sparse data constraints through the identification of patterns and relationships. On the other hand, classification models, such as LightGBM, are less personalised and their focus deviates from novelty exploration. Indeed, they also differ in output format since classification models predict the likelihood of each user to buy each possible item while RSs give a recommendation of which cars fits the best for a given user.

The decision between these alternatives is consistently determined by the particular

objective of the project at hand. In this particular scenario, a Recommender System based on Deep Neural Networks (DNN) is going to be developed, allowing for a performance comparison with the current LightGBM model. Subsequently, the model that shows superior performance and aligns more effectively with the project's goals will be selected as the preferred option moving forward.

5.2 Model description

Recommender Systems pursue the estimation of a value called 'rating'. This value represents the interest and affinity of a certain user for a particular item. Following this logic, those products that obtain the highest predicted ratings will be recommended. In the situation being addressed, there is no concrete data available to indicate user preference for each car, such as surveys or web visits. Therefore, an approach that links the ratings to the occurrence of previous purchases of a particular car by the user has been adopted.

Through this approach, a link is established between purchase frequency and car appreciation. If a user has purchased a car model on multiple occasions, it is inferred that he or she feels a high degree of affinity for that specific model. Following this line of thinking, the rating will be significantly high for such cases, as it is assumed that a recommendation around that car is highly relevant to that user.

Let $y^{(i,j)} \in \mathbb{R}$ denotes the rating defined for the user i to the item j . For instance, given an user i that has bought an A class hatchback ($j = 4$) but not an E class sedan ($j = 12$), the ratings will be $y^{(i,4)} = 1$ and $y^{(i,12)} = 0$. The hybrid filtering mechanism used to predict these ratings, from the user and item data, is formulated as follows (Ng, 2023):

$$\hat{y}^{(i,j)} = v_u^{(i)} \cdot v_m^{(j)},$$

where the elements involved in the model formula are described in Table 5.1.

Table 5.1

Model notation.

Notation	Description
$y^{(i,j)}$	scalar; rating defined for user i on item j
$\hat{y}^{(i,j)}$	scalar; predicted rating for user i on item j
n_u	number of users
n_m	number of items
x_u^i	vector of user i features
x_m^j	vector of item j features
v_u^i	latent vector computed from x_u^i , captures user preferences
v_m^j	latent vector computed from x_m^j , captures item preferences
Y	rating vector storing the ratings $y^{(i,j)}$, $a(n_m \times n_u, 1)$

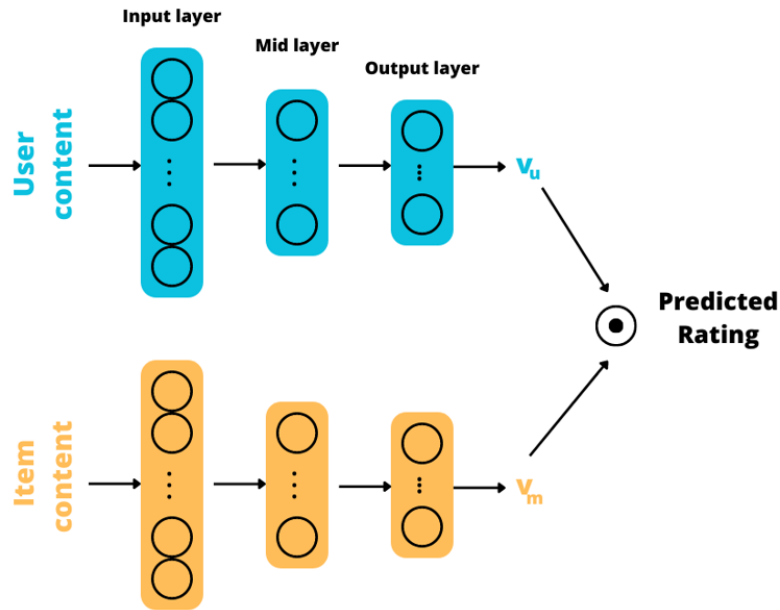
5.3 Model architecture

The method used to develop the hybrid filtering Recommendation System is based on the use of Deep Neural Networks (DNNs). These neural networks consist of several layers in order to capture complex patterns in both user behaviors and item characteristics, thus allowing the generation of accurate and personalized recommendations.

Two neural networks have been created, one of these is in charge of condensing and capturing user preferences, while the other focuses on doing the same for items. This condensation process involves transforming the initial feature vectors of users (x_u) and vehicles (x_m) into more compact and representative vectors: v_u for user preferences and v_m for product patterns. At the end of the process, the output vectors generated by both networks will be combined to make rating predictions (Ng, 2023).

Figure 5.3

Model architecture schema.



The neural networks have been implemented with a dense three-layer structure (see Figure 5.3). A distinctive feature of dense layers is their intensive interconnection. Each neuron in one layer is connected to all neurons in the previous layer, enabling the network to capture complex patterns in the input data.

The composition of these layers is distinguished by two fundamental aspects: the number of neurons that compose them and the activation function that each neuron performs. On the one hand, the number of neurons in each layer determines the complexity and the capacity of the network to understand relationships in the data. On the other hand, the activation functions define the behavior of each neuron.

The number of units, neurons, in each layer will be considered as an adjustable hyperparameter. To explore different configurations, several combinations will be examined.

However, there is one important constraint: the output layers for both users and items must share the same output vector size, which is essential to perform the dot product calculation.

Regarding the activation functions, each layer will have a specifically defined one. The input and mid layers will use the Rectified Linear Activation Function (RELU) which has the following mathematical expression:

$$f(x) = \max(0, x).$$

In this formula, x represents the input to the neuron and $f(x)$ denotes its output. Thus, if x is a positive value (greater than zero), $f(x)$ takes on the value of x . In this case, the neuron is activated and transmits the input as output. If x is a negative value or zero, $f(x)$ takes on the value of zero. In this instance, the neuron is "deactivated" and does not output a signal. This behavior introduces nonlinearity into the network.

In contrast, the output layer will have a linear activation function, implying that the output of this layer is simply composed as a linear combination of the inputs, without involving any nonlinear transformation. This approach allows the network to generate linear predictions based on the features captured by the previous layers.

Once neural networks take the input data (user and item features) and process it through the layers, generating predicted ratings using the dot product, those predicted ratings $\hat{y}^{(i,j)}$ are then compared to the actual ratings $y^{(i,j)}$. The function that measures the discrepancy between the prediction and the actual value is known as the loss or the cost function, which will be implemented as the mean square error (MSE).

$$\text{Cost Function: } J = \frac{1}{n_u \times n_m} \sum_{(i,j)} (\hat{y}^{(i,j)} - y^{(i,j)})^2$$

The aim of the DNN mechanism is to reduce as much as possible this cost function tuning the weights and parameters of the network, that is the reason why an optimizer comes into play. In particular, the Adam optimizer, Adaptive moment estimation, is going to be used. This algorithm uses the concept of gradient descent to update the weights and biases in a direction that reduces the cost function.

These update steps are repeated iteratively throughout the training process. A specific number of training cycles, known as epochs, are established, which determine how many times the weights and network parameters will be adjusted. Each iteration allows the model to be gradually refined, refining its ability to generate more accurate predictions.

The integral development of the recommendation system described above has been materialized through the use of the Keras TensorFlow library in the PySpark environment. A detailed analysis of the components and the specific implementation is available in the appendix of the document.

5.3.1 Integration of data and model input

Once the architecture of the model has been elaborated, it is essential to describe the input requirements to works effectively. This involves describing the structure of the input data set to the deep learning model.

The input is composed of three matrices, which will be named as "user vector" (x_u), "item vector" (x_m) and "rating vector" (Y). These matrices are presented below: (Ng, 2023)

$$\begin{pmatrix} User_ID & Feat_a & \dots & Feat_u \\ 1 & f_{11}^a & \dots & f_{11}^u \\ 1 & f_{12}^a & \dots & f_{12}^u \\ \dots & \dots & \dots & \dots \\ 1 & f_{1n_m}^a & \dots & f_{1n_m}^u \\ \dots & \dots & \dots & \dots \\ n_u & f_{n_u1}^a & \dots & f_{n_u1}^u \\ n_u & f_{n_u2}^a & \dots & f_{n_u2}^u \\ \dots & \dots & \dots & \dots \\ n_u & f_{n_un_m}^a & \dots & f_{n_un_m}^u \end{pmatrix}, \begin{pmatrix} Item_ID & Feat_b & \dots & Feat_m \\ 1 & f_{11}^b & \dots & f_{11}^m \\ 2 & f_{12}^b & \dots & f_{12}^m \\ \dots & \dots & \dots & \dots \\ n_m & f_{1n_m}^b & \dots & f_{1n_m}^m \\ \dots & \dots & \dots & \dots \\ 1 & f_{n_u1}^b & \dots & f_{n_u1}^m \\ 2 & f_{n_u2}^b & \dots & f_{n_u2}^m \\ \dots & \dots & \dots & \dots \\ n_m & f_{n_un_m}^b & \dots & f_{n_un_m}^m \end{pmatrix}, \begin{pmatrix} Rating \\ y^{(1,1)} \\ y^{(1,2)} \\ \dots \\ y^{(1,n_m)} \\ \dots \\ y^{(n_u,1)} \\ y^{(n_u,2)} \\ \dots \\ y^{(n_u,n_m)} \end{pmatrix}$$

where f_{ij}^k refers to the value of the feature k associated with the $User_ID = i$ and the $Item_ID = j$, and $y^{(i,j)}$ denotes the rating defined for the user i to the item j .

The "user vector" gathers the user information. There exist n_m rows per client, one for each possible item, such that they hold the characteristics related to the user with the corresponding item. For instance, one of those features is the age at which each client made the purchase. The features selected for "user vector" are listed in table 5.2.

The "item vector", on the other hand, gathers information related to the vehicle. Then, n_u rows per item are arranged, one for each user, to capture the individual characteristics of each item depending on the particular configuration of each user. For example, one of those features is the price that the client paid for the vehicle. The features selected for "item vector" are listed in table 5.2.

The "rating vector" collects the ratings defined for each user to each item. For instance, given an user i that has bought an A class hatchback ($j = 4$) but not an E class sedan ($j = 12$), the ratings will be $y^{(i,4)} = 1$ and $y^{(i,12)} = 0$.

Since the model architecture is characterized by its high flexibility and efficiency, allowing it to address situations where classes are unbalanced or underrepresented, it was decided to go deeper in the prediction process. Instead of items being described only by class and body type, level BM4, the approach has been extended to include levels of detail up to BM6, which involves predicting not only class and body type, but also engine type, fuel and drive system. In this way, the number of elements n_m to be predicted is adjusted to the number of combinations available at the BM6 level. Following the previous example, if user i has bought an A class hatchback, with engine code 180, fuel type petrol and

4x2($j = 4$) but not an E class sedan, with engine code 300, fuel type diesel and 4matic ($j = 12$), the ratings will be $y^{(i,4)} = 1$ and $y^{(i,12)} = 0$.

Table 5.2

*Model features. The ones marked with an asterisk * indicate that there are supplementary features bearing the same name but appended with distinct attributes represented by the asterisk. On the one hand, mileage_median_q* measures the median mileage covered by the car during a specific quarter of the year. Indeed, there are 4 features: mileage_median_q1, mileage_median_q2, mileage_median_q3 and mileage_median_q4, reflecting the median mileage for each quarter. On the other hand, the asterisk in the remaining features represents the median, the mean and the standard deviation of each of them. For instance, the median_damage_total_request_amount describes the median price of the damages repairs the car had.*

User features	Item features	
ciam_id	BM6_index	model_rank
age_at_purchase	Active_status	time_active
city	class_type	body_type
role	engine_type	fuel_type
gender	drive_type	total_price
financial_product_group	list_price_without_sa	list_price_of_all_sa
finance_leasing_status	*num_damages_per_service	*total_num_services
amg_type	mileage_median_q*	*num_warranty_repairs
maybach	*damage_total_request_amount	

As a result, the integration and organization of the available data to fit the previously detailed structure is required. To do so, a baseline matrix including all possible combinations between customers and vehicles (Baumuster 6) is generated. Then, the rating for each combination will be computed, depending on whether or not the customer has purchased the model in question.

The following step is to incorporate the customers' and vehicles' information into the base matrix. In this way, the customer-vehicle combinations that occur in reality, that is, the rows corresponding to the vehicles purchased by that individual, will be enriched with the corresponding information, for instance, the age at which the customer bought the vehicle and the price paid. In cases where an individual has purchased the same vehicle model on several occasions, only the data from the most recent model will be recorded.

Finally, the gaps in the information related to those combinations that don't occur in reality will be filled in by default values. This means that default values will be assigned to the characteristics of the rows corresponding to vehicles that each user has not purchased. Consequently, the resulting matrix will comply the deep learning architecture standards, therefore, the three necessary inputs for the DL architecture will be extracted by choosing the relevant columns for each.

5.3.2 Hyper-parameters

Hyperparameters are not parameters that the model learns during training, but rather they are pre-settings that influence how the optimization and modeling of the data is performed. These hyperparameters guide the behavior of the network during the training process, affecting the speed of convergence, generalization capability and efficiency in finding optimal solutions (Goodfellow et al., 2016).

The main hyperparameters on which tests have been carried out with different values are presented below.

- **Batch size:** Number of training examples used in each iteration of the algorithm's optimization process. Instead of updating the model's parameters after processing the entire dataset (batch size equals the dataset size), training is often performed in smaller subsets or batches. These batches allow for more frequent updates of the model's parameters, which can lead to faster convergence and more efficient utilization of computational resources. Search space explored: [16,32,64].
- **Epoch:** A single pass of the entire training dataset through the neural network. During one epoch, the algorithm uses the entire dataset to update the model's parameters (weights and biases) in order to minimize the chosen cost function. The number of iterations required to complete one epoch depends on the batch size. If the dataset has N examples and the batch size is B , then the number of iterations needed to complete one epoch would be N / B . This hyperparameter offers the possibility of adjusting the balance between the time needed for training and the improvement in predictions. Search space explored: [30,40].
- **Learning rate:** The size of the steps the optimizer takes in the weight adjustment process. It plays a pivotal role in determining the convergence behavior of the optimization algorithm. A learning rate that is excessively high can lead to optimization divergence, causing the algorithm to overshoot and fail to reach the optimal solution. Conversely, an excessively low learning rate can impede the optimization process by significantly slowing down the convergence rate. Search space explored: [0.001,0.005, 0.01, 0.05].
- **Layer units:** The number of neurons or units present in a particular layer of a neural network. In a neural network architecture, each layer consists of interconnected neurons, and the number of units in a layer determines the complexity and capacity of that layer to capture patterns and features in the data. Increasing the number of units in a layer increases the model's capacity to learn complex patterns in the data. However, too many units can lead to overfitting, where the model learns noise in the training data rather than general patterns. Search space explored for (*input_layer_cells_users*, *mid_layer_cells_users* and *num_outputs_user*) and

(*input_layer_cells_items*, *mid_layer_cells_items*, *num_outputs_items*): [(256, 128, 64), (256, 128, 32), (128, 64, 32)] .

5.4 Negative sampling approach

In a Recommender System, the main objective is to predict which items would be of interest to a user based on his or her history and preferences. However, in situations where the number of potential items is extremely large, generating all possible user-item combinations for training is impractical and computationally expensive. For example, in the case of the Italian market, the model input matrices consist of about 41 million rows, while in even larger markets, such as Germany, the figure amounts to approximately 328 million rows. The sheer magnitude of these datasets leads to computational times that are suboptimal, and, at times, unfeasible for the specific purpose of our use case. Therefore, it is crucial to explore techniques that address these computational challenges effectively.

In this context, the widely used approach to address this situation in Recommender Systems is known as ‘Negative Sampling’ (Rawat et al., 2019; Pennington et al., 2014). Instead of considering all possible combinations, the algorithm selects the items that the user has interacted positively (i.e. ratings greater than 0) and a small subset of items that the user has not interacted with (i.e. ratings equal to 0), known as ‘negative items’.

Most customers have not interacted with more than 3 different car models, or there is no record of it, so positive items represent less than one percent of the rows for each customer. Thus, by focusing on a representative subset of negative items, Negative Sampling drastically reduces computational complexity, which in turn accelerates model training speed (Daghaghi et al., 2020).

However, the choice of negative elements is critical to the success of the technique, it must be carefully balanced. In particular, the selected strategy is based on selecting negative examples that share certain similarities with the elements that the client has previously acquired and including randomly selected negative examples. This selection approach contributes to enriching the methodology used in the process.

On the one hand, the choice of negative examples that present similarities with those items that the user has positively acquired lies in the assumption that if a user liked a specific item, it is plausible that he or she will also be interested in other similar items. This is intended to enrich the diversity of recommendations and not be limited only to the user’s already known preferences.

On the other hand, randomly chosen negative examples prevent the model from becoming too biased, in other words, it avoids overfitting. Incorporating random examples prevents the model from focusing only on already known preferences and instead gives it the ability to explore new recommendation possibilities.

To compute the similarities between vehicles, a distance matrix should be computed.

However, the first step will be to choose a proper distance. Despite the elements of the matrix are actually numbers, some of them are index-linked to categorical data. As a result, the distance selected is the Gower distance, a metric that considers the difference in the nature of the data and applies similarity measures appropriate for each type of variable. For categorical variables, it uses a measure of coincidence, and for numerical variables, it uses the Euclidean distance.

The Gower's distance between two vectors, A and B, with numerical and categorical data is $\delta_{AB}^2 = 1 - s_{AB}$, where

$$s_{AB} = \frac{\sum_{h=1}^{m_1} (1 - |x_{Ah} - x_{Bh}|/G_h) + \alpha}{m_1 + m_2}$$

is the Gower's similarity coefficient, m_1 is the number of numerical variables, m_2 is the number of categorical variables, α is the number of matches in the categorical variables, G_h is the range for the variable h and x_{Ah} represents the value of the column h for the A vector (Gower, 1966).

From this point, a criterion is established based on the customer's purchase history to select the negative rows in a specific way:

If a customer has purchased only one combination of BM6, for instance, he had only bought *c_class_coupe_220_p_4x2* models, the three top rows corresponding to items similar to the one purchased are added. In the case of customers who have purchased two combinations of cars, the top two rows of similar items are added for each model the customer has purchased. Moreover, when a customer has purchased three or more combinations of cars, the top 1 similar item is added for each combination the customer has purchased. Finally, two random items from the ones that have not been previously selected are also added.

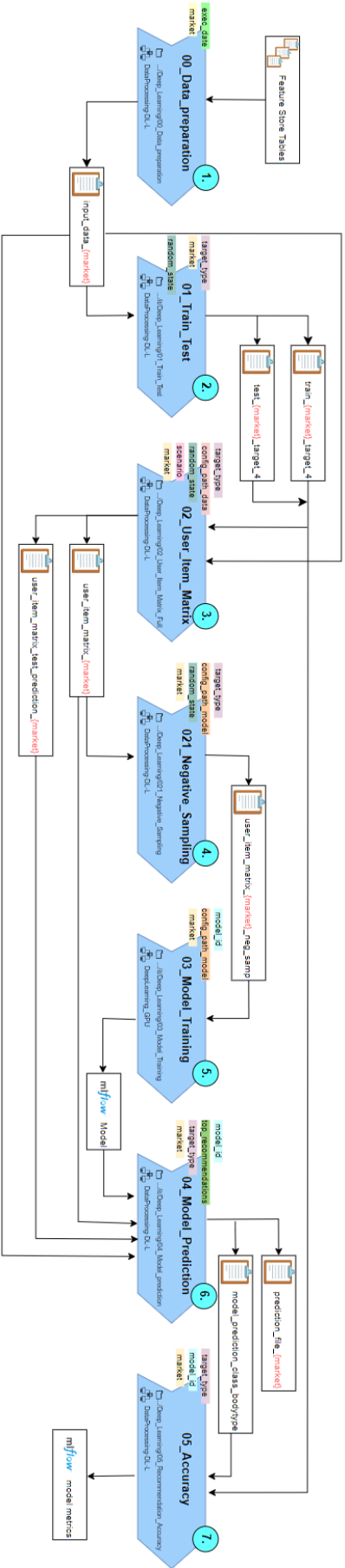
By combining similar examples with random examples, a balance is achieved that reflects both known preferences and possible unknown preferences of the user. This helps to avoid saturating the model with already known information and instead encourages diversity and originality in the recommendations generated.

5.5 Workflow Implementation

The previous sections have detailed the components and procedures designed for the hybrid filtering Recommendation System. However, it still misses to state the workflow followed in order to illustrate how these previously described tools have been integrated.

Seven python notebooks have been created and nested into a Databricks pipeline to deploy and automate the predicting process. This workflow is illustrated by the Figure 5.4, where the blue shapes refer to python notebooks, the arrows indicate their input and output and the colored tags on the left-hand side denote the input variables that should be pre-defined.

Figure 5.4
Model workflow.



1. The workflow starts importing the tables stored in Feature Store into the *00_Data_preparation* notebook. This data has been already preprocessed, therefore, the first step aims to join the cleaned tables into a single database (at a customer-vehicle level), create additional columns to summarize the data from tables that contain event-based information and select the features listed in Table 5.2. The resulting data frame is then stored in a delta table called *input_data_{market}*, where *{market}* refers to the country code of the particular data, for instance, *input_data_it* gathers Italian information.

2. The *01_Train_Test* notebook takes the resulting file of the previous notebook to divide the rows into train and test sets. However, the choice of the split should be made with caution, as the data includes sequential information and classes that are underrepresented. Thus, if a random split takes place, there is a significant risk of obtaining biased results and experiencing data leakage. As a consequence, the split takes place by selecting the last purchase of each customer and performing a stratified sampling within those. In this way, 20% of the last purchases, conserving the proportion of BM6 codes, are stored to test the algorithm. The remaining ones, along with the customer purchase history, will be used to train the model.

An illustrative example of the train-test split is provided in Table 5.3. Considering two customers of the *input_data_it* data frame, the last purchase of each is highlighted in bold. From them, let's assume that the one with grey background is selected for testing by the stratified sampling. Then, the grey row is stored in *test_it_target_4* and the remaining ones are stored in *train_it_target_4*.

Table 5.3

Illustrative example of the rows' split into train and test.

<i>ciam_id</i>	<i>BM6</i>	<i>valid_from</i>
00ab79e	<i>a_class_hatchback_180_p_4x2</i>	2016-12-05
00ab79e	<i>gla_class_suv_180_p_4x2</i>	2020-03-24
00ab79e	<i>gla_class_suv_200_p_4x2</i>	2023-05-01
00df18b	<i>glc_class_coupe_220_d_4matic</i>	2018-04-20
00df18b	<i>glc_class_suv_220_d_4matic</i>	2023-04-20

3. Data should be transformed to fit the model input requirements (see section 5.3.1). This step is taken by the *02_User_Item_Matrix* notebook, where the complete matrix with all possible combinations of users and items (*user_item_matrix_it*) is created based on the training set (*train_it_target_4*). Considering the same situation exemplified before, Table 5.4 illustrates how the *user_item_matrix_it* looks like for the customer with *ciam_id* = 00ab79e.

Table 5.4

Illustrative example of the user_item_matrix for a particular user, ciam_id = 00ab79e. Due to the gla_class_suv_200_p_4x2 was selected to test the algorithm, this row should only contain default values, exactly as if the customer had not bought that car.

<i>ciam_id</i>	<i>BM6</i>	<i>age_at_purchase</i>	<i>...</i>	<i>rating</i>
00ab79e	<i>a_class_hatchback_180_p_4x2</i>	30	...	1
00ab79e	<i>gla_class_suv_180_p_4x2</i>	34	...	1
00ab79e	<i>gla_class_suv_200_p_4x2</i>	-99	...	0
...	...	...	...	
00ab79e	<i>glc_class_coupe_220_d_4matic</i>	-99	...	0
00ab79e	<i>glc_class_suv_220_d_4matic</i>	-99	...	0

Moreover, this notebook also creates a matrix, called *user_item_matrix_test_prediction_it*, in order to store the information for those combinations of users and items selected to test the algorithm. This data could not be included to train the model because they will be used to test the results, however, they must be included to generate the final predictions. This is the reason why they should be stored. In the provided scenario, Table 5.5 illustrates the test row stored for the illustrative customer.

Table 5.5

Illustrative example of the user_item_matrix_test_prediction for a particular user, ciam_id = 00ab79e. It contains the rows storing the information for testing.

<i>ciam_id</i>	<i>BM6</i>	<i>age_at_purchase</i>	<i>...</i>	<i>rating</i>
00ab79e	<i>gla_class_suv_200_p_4x2</i>	37	...	1

4. The *user_item_matrix* is then ready to train the model, nevertheless, to avoid impractical computational times, a negative sampling technique, explained in section 5.4, is applied in the *021_Negative_Sampling* notebook. Afterward, the resulting matrix (*user_item_matrix_it_neg_samp*) will be the input to train the DNN Recommender System.

5. *03_Model_training* notebook starts dividing the matrix *user_item_matrix_it_neg_samp* into the user, the item and the rating matrices, selecting the corresponding columns. Subsequently, the features are standardized using the `StandardScaler()` function for user and item features and `MinMaxScaler((-1, 1))` for ratings, both of the `sklearn` python module. Afterward, the model is trained using the implementation detailed in the Appendix. There, once the model has been trained, it is registered in MLflow, together with parameters and additional information such as the run time, using an identification number, *model_id*. For a thorough comprehension of the Deep Learning model, please refer to section 5.3, where an in-depth description of its architecture is provided.

6. The *04_Model_Prediction* notebook starts loading the model from MLflow and importing the matrix *user_item_matrix*. Thereafter, the model predicts the ratings for the matrix, storing the results in *model_prediction_class_bodytype*, and the greatest rating for each user will be the recommended item. Nonetheless, it is important to remark that the predictions generated for testing users cannot be considered final predictions. This is because they have been generated without taking into account the last purchase information. Then, those predictions will only be used to test the algorithm in the final notebook *05_Accuracy*.

The final prediction file, the one that will be sent with the recommended items once the model's accuracy is approved, is also built in the *04_Model_Prediction* notebook. Final predictions of users who have not been selected for testing are already generated. Then, once the final predictions for the test users are made, these predictions are combined.

To generate test users' final predictions, the *user_item_matrix* is updated with the data from *user_item_matrix_test_prediction* and these users are re-predicted with the updated matrix. The particular updated matrix for the customer serving as example, *ciam_id* = 0079abe, could be found in Table 5.6. These predictions, which take into account the information from the latest purchase, are merged with the final predictions for the training users and they collectively form the *prediction_file_it*.

Table 5.6

Illustrative example of the updated matrix, combining user_item_matrix and user_item_matrix_test_prediction, to generate a final prediction for the particular test user with ciam_id = 00ab79e.

<i>ciam_id</i>	<i>BM6</i>	<i>age_at_purchase</i>	...	<i>rating</i>
00ab79e	<i>a_class_hatchback_180_p_4x2</i>	30	...	1
00ab79e	<i>gla_class_suv_180_p_4x2</i>	34	...	1
00ab79e	<i>gla_class_suv_200_p_4x2</i>	37	...	1
...	...	...	...	
00ab79e	<i>glc_class_coupe_220_d_4matic</i>	-99	...	0
00ab79e	<i>glc_class_suv_220_d_4matic</i>	-99	...	0

7. *05_Accuracy* notebook imports the predictions generated for the test users, not the final ones but those that were stored in *model_prediction_class_bodytype*, and compares them with the last users' purchase. The evaluation criteria involved to conclude how well the model performs is described in chapter 6, and those metrics will be also registered in MLflow associated with the same *model_id*.

Evaluation and results of the novel approach

6.1 Performance metrics and evaluation criteria

Once again, the results achieved for the Italian market are the ones presented, they reflect the predominant behavior of the model in the European countries as a whole. The optimal configuration of the hyperparameters was found to be batch size = 64, learning rate = 0.001 and epochs = 40, while the layer units for users were set to (256, 128, 64) and for items to (256, 128, 64).

It is essential to highlight that, although the model makes predictions at the Baumuster 6 level, an exhaustive analysis of the metrics at different levels has been performed. This has been done with the purpose of addressing the behavior of the model in the present situation (class and body type predictions) and in possible future extensions (BM6). Furthermore, the F1 weighed and the runtime are also taken into account, which are detailed in the Table 6.1.

In addition, the performance of a reference model, called the "naive model", has been incorporated in order to establish a robust comparison and understand whether the proposed Recommender System actually provides significant improvements in recommendation accuracy. The naive model's approach is to predict exactly the same vehicle that users already own today. This inclusion makes possible to determine whether the data follow a naive pattern of selection, regardless of user preferences and similarity between vehicles.

Table 6.1*Model metrics for Italian data.*

Metrics	Naive	DNN	DNN + neg sampling
Class_Bodytype accuracy	Top1: 30.5%	Top1: 31.0%	Top1: 31.1%
	Top2: 30.5%	Top2: 34.0%	Top2: 38.0%
	Top3: 30.5%	Top3: 35.3%	Top3: 43.9%
Class_Bodytype_engine accuracy	Top1: 17.0%	Top1: 17.2%	Top1: 17.1%
	Top2: 17.0%	Top2: 18.5%	Top2: 20.1%
	Top3: 17.0%	Top3: 19.3%	Top3: 20.6%
BM6 accuracy	Top1: 12.9%	Top1: 13.1%	Top1: 12.9%
	Top2: 12.9%	Top2: 14.2%	Top2: 15.1%
	Top3: 12.9%	Top3: 14.9%	Top3: 15.7%
F1 weighed	0.295	0.299	0.300
Runtime	-	60min	2min

In the first place, the significant reduction in computational time achieved by implementing the negative sampling technique in the Recommender System is the most remarkable result of the table. In turn, this time improvement is accompanied by results that match those obtained by the simple Deep Neural Network model (without negative sampling technique). Moreover, analyzing the first three recommendations, the significant improvement introduced by the application of negative sampling becomes particularly evident. Consequently, the use of this technique in conjunction with the DNN model will be established as the alternative approach to compare with the current model based on LightGBM.

Secondly, a gradual decrease in accuracy is observed as the prediction is expanded to more vehicle-specific elements, particularly from a BM4 prediction to BM6. This decrease in accuracy is due to the increase in the possible existing combinations, given the greater complexity of the details to be considered.

Finally, it is relevant to point out the similarity between the results of the naive model and those referring to the implemented recommendation model. This highlights the lack of distinctive patterns in the data, with a common tendency for users to purchase the same model they previously owned 30.5% of the time. However, while the recommender improvement may seem modest, it is significant given that a one percent increase in accuracy results can translate into gains of several thousand euros for the brand.

6.2 Comparison with the existing model

To compare the deep learning Recommender System with the LightGBM classification model, the corresponding metrics have been analyzed and compiled in Table 6.2. This

table reflects the prediction evaluations at the level of classes and body types (BM4).

Table 6.2

Comparison metrics for Italian data.

Metrics	LightGBM	DNN + neg sampling
Class_Bodytype accuracy	Top1: 26.1%	Top1: 31.1%
	Top2: 39.3%	Top2: 38.0%
	Top3: 48.9%	Top3: 43.9%
F1 weighed	0.247	0.300
Runtime	4min	2min

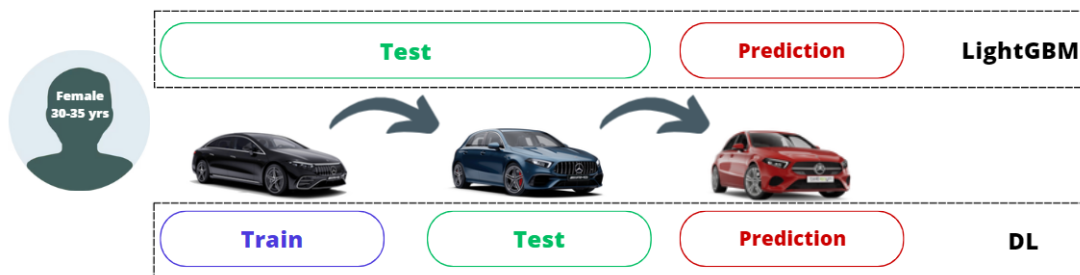
Examining the results for the Italian market, in terms of computational time, it has been observed that the deep learning model proves to be more efficient. Although the difference in time may not seem significant in a first analysis, it becomes crucial when considering implementation in larger markets, such as Germany.

Moreover, the deep learning model shows a higher weighted F1 score. This implies that the deep learning model has a higher ability to accurately predict minority classes compared to the LightGBM model.

Finally, a higher accuracy value for the first recommendation is achieved employing the deep learning approach. However, considering the second and third recommendations, the accuracy evens out and, in some cases, the LightGBM-based model outperforms the deep learning model. However, it is relevant to mention that the one-to-one comparison could not be fairly made because the train-test split has been done in a different way. On the one hand, the lightGBM accuracy measures how well the model extrapolates the patterns learned for the train customers to unknown ones, predicting for each purchase row the next best car. On the other hand, deep learning accuracy measures how well the model learns general patterns and purchase preferences for a given user, predicting the next best car for each known customer.

Figure 6.1

Illustrative example of how the different approaches treat a testing customer.



A hypothetical example to illustrate the testing differences between both solutions is provided by Figure 6.1. Let's assume this female is selected for testing. On the one hand, the Lightgbm model has no previous information about the customer, and the two rows

kept for testing are going to be predicted. In this way, if the model recommends for the row with the black vehicle the blue one, a match will be considered, in the same vein, if the model predicts for the blue vehicle the red one, another match will be counted. On the other hand, the Recommender System has the information that the customer already purchased the black vehicle and it predicts using the information of the blue one. If the recommendation is the red vehicle, then a match will be considered. As a consequence, Lightgbm model had 2 opportunities to predict correctly while the DL only had one for the same customer.

6.3 Strengths and limitations of the model

One of the most notable advantages of using the deep learning approach, supported by the negative sampling technique, lies in its low computational cost. It has been observed that even for large markets, the training process can be completed in as little as 16 minutes. This level of training efficiency allows for effective scalability of the model, which is particularly relevant when dealing with larger and more complex market environments.

Flexibility is another outstanding advantage of this Recommender System. It is capable of making predictions up to BM6 level, making it a versatile resource for various levels of detail. Besides, it has the ability to incorporate new features easily and with reduced computational cost.

The train-test split configuration is also an outstanding key point of the solution. The algorithm learns previous user preferences as well as general patterns to predict the next purchase of those users. As a result, it provides a reliable picture of the model performance using the predictions that will be delivered. Moreover, it is also capable of handling customers with only 1 purchase because the target column is the car rating, whether the customer has bought a particular car, therefore, more data can be used to train and evaluate the model.

However, the deep learning-based Recommender System is not without limitations. Although it shows considerable training time efficiency and offers significant flexibility, the metrics in terms of accuracy do not substantially exceed those provided by the LightGBM model. These accuracy results below what could be expected may be attributed, at least in part, to the limited availability of item features. The model lacks detailed information regarding intrinsic vehicle characteristics, such as size, weight and color, among other aspects. This lack could be affecting its ability to capture more accurate and sophisticated relationships between user preferences and specific vehicle properties.

Finally, it is remarkable to point out that its schema does not allow to use Feature Store capabilities, at least in a simple way as LightGBM does. As a consequence, more storage space is needed to store the joined tables and not having to do the table joins manually each time the predicting process takes place.

Conclusions

LightGBM algorithm is the main character of the NBx use case, predicting when the next customer's purchase will take place and which class and body type will be bought. The current results are pretty competitive from a business point of view, in particular, taking into account this is a real-life scenario involving a real database. An additional key point is the usage of a novel tool implemented by Databricks, called Feature Store, that greatly simplifies the predicting process by selecting and combining the features that are used in the model trained.

Nevertheless, there is still room for improvement since the existing approach requires significant computational resources for large data, such as the German market, and for a deeper predicting process, such as BM6 predictions. Moreover, the training and evaluation sets are limited due to customers with only one purchase could not be considered and there is a high metrics' dependency on the hyperparameters, revealing the importance of the hyperparameter tuning process.

A Recommender System based on Deep Neural Networks (DNNs) has been explored and implemented, providing an effective alternative for the next best car prediction task. Indeed, the algorithm is able to predict at BM6 level while requiring only half processing time. The results are pretty similar to the ones provided by the LightGBM model, however, the one-to-one comparison could not be fairly made due to the different designs in the evaluation set. In fact, the approach followed by the Recommender System gives more reliable insights into the model performance.

Additional key points of the novel approach are the flexibility and high performance to address minority classes and customers with only one purchase, and the sampling technique called Negative Sampling that reduces the model input size as well as enhances the metrics. However, it should be noted the lack of vehicle specifications in the data, which

could be a limiting factor detecting patterns within the user preferences and computing similar vehicles. Certainly, it could also be the reason why the deep learning metrics don't excel in the analysis.

Overall, both the LightGBM-based solution and the DNN-based Recommender System come with their own set of advantages and disadvantages. Despite a one-to-one comparison has not been feasible on entirely fair conditions, the deep learning solution has been designed to take predictions a step further and aligns more closely with the current and future product requirements. Consequently, the NBx team has decided to invest in this new alternative, although further explorations should be still made to compare both accuracy results.

7.1 Future lines of work

There exist several steps that should be still taken to fairly compare LightGBM and DNN approaches. One of the topics being discussed is the real-time accuracy measurement. The proposal campaign is based on dividing the users in two groups per market, predicting one of them with LightGBM model and the other with the deep learning solution, and sending the corresponding predictions. Afterward, tracking the purchases of customers who has actually bought a car, an overview of how good the model results are and which of them perform best could be obtained.

Another idea to continue the analysis and improve the use case results is to request additional characteristics related to the vehicle specifications. In this way, the model architecture previously implemented could find more patterns and distinguish better the similarities and differences between vehicles.

Furthermore, new neural networks implementations in order to take into account the order of the customers' purchases could be explored, aiming to capture the sequential patterns in the data. As a result, not only purchases could be included, also the entire event-based data could be used for training. Some ideas to follow up the exploration will be the deployment of common approaches such as Recurrent Neural Networks (RNNs) or the groundbreaking Transformer-based models designed for sequences. Transformers use an attention mechanism that enable them to process input data in parallel rather than sequentially, making them highly efficient for both training and inference.

Bibliography

- Daghaghi, S., Medini, T., Meisburger, N., Chen, B., Zhao, M., & Shrivastava, A. (2020). A tale of two efficient and informative negative sampling distributions. *arXiv preprint arXiv:2006.10930*.
- Databricks. (2023). *Databricks machine learning feature store documentation*.
- Europa-Press. (2023). Mercedes benz gana 7.500 millones hasta junio, un 13,8% más. *Europa Press*. <https://www.europapress.es/motor/sector-00644/noticia-mercedes-benz-gana-7500-millones-junio-14-mas-20230727101540.html>
- Friedman, J., Hastie, T., & Tibshirani, R. (2001). *The elements of statistical learning*. Springer Series in Statistics.
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.
- Gower, J. C. (1966). Some distance properties of latent root and vector methods used in multivariate analysis. *Biometrika*, 53(3/4), 325–338. <https://doi.org/10.2307/2333639>
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., & Liu, T.-Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. *Advances in Neural Information Processing Systems*, 30.
- Microsoft LightGBM Contributors. (2023). Lightgbm documentation.
- Ng, A. (2023). *Unsupervised learning, recommenders & reinforcement learning* [Offered by DeepLearning.AI and Stanford University.]. <https://www.coursera.org/learn/unsupervised-learning-recommenders-reinforcement-learning?specialization=machine-learning-introduction>
- Pennington, J., Socher, R., & Manning, C. D. (2014). GloVe: Global vectors for word representation. *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 1532–1543.
- Portugal, I., Alencar, P., & Cowan, D. (2018). The use of machine learning algorithms in recommender systems: A systematic review. *Expert Systems with Applications*, 97, 205–227. <https://doi.org/https://doi.org/10.1016/j.eswa.2017.12.020>
- Rawat, A. S., Chen, J., Yu, F. X. X., Suresh, A. T., & Kumar, S. (2019). Sampled softmax with random fourier features. *Advances in Neural Information Processing Systems*, 13834–13844.

Appendix: Recommender System Implementation

The hybrid filtering Recommender System has been built based on the examples provided in the Stanford course Ng, 2023 and the configuration detailed in section 5.3. Deep neural networks are the main character of the implementation, which has been easily coded by the Python library Keras. It provides high-level access through an API to the functionalities of another library called TensorFlow, used for the low-level creation of deep learning models. The combination of both allows building deep neural networks just specifying all the elements that compose it, from the structure of the layered network to the activation functions, evaluation criteria and other functionalities. Further information can be found in the official documentation of the mentioned libraries.

The core code of the Recommender System is not considered confidential information, therefore, it is provided below together with explanatory comments to enhance its understanding. Furthermore, the integration of the model into MLflow is also detailed. This particular step holds significant importance as it simplifies the tracking of the model's lifecycle and facilitates future reference and consultation.

In the first place, Code .1 represents the designing of the model architecture described in section 5.3. There are some variables that should be previously defined: the number of neurons per layer (*input_layer_cells_users*, *mid_layer_cells_users* or *num_outputs_user*), which have been established by the hyperparameter optimization, the number of user and item features and the random state.

CODE .1. Designing the Recommender System architecture: item and user deep neural networks plus the dot product between the outputs

```
1  import tensorflow as tf
2  from tensorflow import keras
3  from tensorflow.keras.callbacks import EarlyStopping
4  from tensorflow.keras import callbacks
5  from sklearn.preprocessing import StandardScaler, MinMaxScaler
6  import mlflow
7
8  tf.random.set_seed(random_state)
9
10 # User DNN formed by 3 layers
```

```

11 user_NN = tf.keras.models.Sequential([
12     tf.keras.layers.Dense(input_layer_cells_users, activation='
        relu'),
13     tf.keras.layers.Dense(mid_layer_cells_users, activation='relu'
        ),
14     tf.keras.layers.Dense(num_outputs_user, activation='linear')
15 ])
16 # Item DNN formed by 3 layers
17 item_NN = tf.keras.models.Sequential([
18     tf.keras.layers.Dense(input_layer_cells_items, activation='
        relu'),
19     tf.keras.layers.Dense(mid_layer_cells_items, activation='relu'
        ),
20     tf.keras.layers.Dense(num_outputs_items, activation='linear')
21 ])
22
23 # create the user input and point to the base network
24 input_user = tf.keras.layers.Input(shape=(num_user_features))
25 vu = user_NN(input_user)
26 vu = tf.linalg.l2_normalize(vu, axis=1)
27
28 # create the item input and point to the base network
29 input_item = tf.keras.layers.Input(shape=(num_item_features))
30 vm = item_NN(input_item)
31 vm = tf.linalg.l2_normalize(vm, axis=1)
32
33 # compute the dot product of the two vectors vu and vm
34 output = tf.keras.layers.Dot(axes=1)([vu, vm])
35
36 # specify the inputs and output of the model
37 model = tf.keras.Model([input_user, input_item], output)

```

In the second place, Code .2 starts an MLflow experiment, trains the model minimizing the cost function by the Adam optimizer and registers it into MLflow together with the corresponding parameters. It is remarkable to note that there are also variables that should be previously defined: the hyperparameters epochs, learning rate and batch size, the configuration of an early stop policy to finish early the training process if it exceeds the allocated time and the model identification. Moreover, the input matrices should be also provided, the user, the item and the Y matrix gathering the ratings.

CODE .2. Training the model and registering it into MLflow

```

1 client = mlflow.tracking.MlflowClient()
2 mlflow.set_experiment(experiment_name) # set the experiment name
3 start = datetime.datetime.now()
4
5 # start the MLflow experiment
6 with mlflow.start_run() as run:
7

```

```

8      # Activate tensorflow autolog
9      mlflow.tensorflow.autolog()
10
11     # Cost Function
12     cost_fn = tf.keras.losses.MeanSquaredError()
13     # Adam optimizer
14     opt = keras.optimizers.Adam(learning_rate=learning_rate)
15
16     # Compile Model
17     model.compile(optimizer=opt,
18                   loss=cost_fn)
19
20     # Early Stop Policy
21     early_stop = EarlyStopping(monitor=monitor,
22                                patience=patience)
23
24     # Fit Model
25     history = model.fit([user_train[:, u_s:],
26                          item_train[:, i_s:]],
27                          y_train,
28                          batch_size=batch_size,
29                          epochs=epochs,
30                          callbacks=[
31                              early_stop,
32                              tensorboard_callback
33                          ])
34
35     runtime = datetime.datetime.now() - start
36
37     # Evaluate model
38     eval_loss = model.evaluate([user_test[:, u_s:],
39                                item_test[:, i_s:]], y_test)
40     mlflow.log_metric("evaluation_loss", eval_loss)
41
42     # Save model also in local path to ADLS
43     model.save(current_model_folder)
44
45     # Log Parameters
46     mlflow.log_params(params)
47     mlflow.log_params({'model_loc':current_model_folder,
48                        'user_features':user_features,
49                        'item_features':item_features,
50                        'runtime': str(runtime)})
51
52     # The timestamp is the model identification number
53     mlflow.set_tag("timestamp", timestamp)
54
55     # Finish the experiment
56     mlflow.end_run()

```